

1. Projektziel

Im Rahmen des Kurses „Objektorientierte und funktionale Programmierung mit Python“ wurde ein prototypisches Dashboard zur Analyse und Visualisierung des persönlichen Studienfortschritts entwickelt. Ziel des Projekts war es, zentrale Kennzahlen des Studiums übersichtlich darzustellen und dadurch eine schnelle Einschätzung des aktuellen Leistungs- und Fortschrittsstandes zu ermöglichen.

Im Mittelpunkt standen zwei konkrete Ziele: Das Studium soll innerhalb von 36 Monaten abgeschlossen werden und der gewichtete Notendurchschnitt soll höchstens 1,7 betragen. Ergänzend dazu werden der ECTS-Fortschritt, die Notenentwicklung, der Lernfortschritt einzelner Module sowie der Fortschritt verschiedener Prüfungsleistungen betrachtet.

2. Vorgehensweise und Modellierung

In der Konzeptionsphase wurden die Ziele des Dashboards definiert, eine Skizze der Oberfläche erstellt und ein fachliches UML-Klassendiagramm entwickelt. Dieses beschreibt die zentralen Entity-Klassen des Systems, darunter Studiengang, Semester, Modul, Lernfortschritt, Prüfung und Note. Zusätzlich wurden die verwendeten Werkzeuge und Technologien ausgewählt und begründet.

In der Erarbeitungs- und Reflexionsphase wurde untersucht, wie sich diese fachlichen Konzepte objektorientiert mit Python umsetzen lassen. Die zentralen Modellklassen wurden als `dataclasses` umgesetzt, da diese eine übersichtliche Definition von Datenobjekten ermöglichen und gleichzeitig Methoden zur Berechnung fachlicher Kennzahlen enthalten können. Ein besonderer Schwerpunkt lag auf der Modellierung der Prüfungsleistungen. Die allgemeine Klasse `Exam` enthält gemeinsame Eigenschaften wie Prüfungsdatum, Status und optionale Note. Spezialisierte Prüfungsarten wie `PortfolioExam`, `HomeworkExam`, `ProjectReportExam`, `CaseStudyExam` oder `BachelorThesisExam` erben von dieser Basisklasse und besitzen jeweils eigene Fortschrittsberechnungen. Dadurch wird Polymorphie genutzt, da die Benutzeroberfläche allgemein mit Prüfungsobjekten arbeiten kann, ohne die konkrete Prüfungsart kennen zu müssen.

3. Technische Umsetzung und Architektur

Der finale Prototyp wurde als terminalbasierte Anwendung mit der Bibliothek `Textual` realisiert. `Textual` wurde gewählt, weil damit eine strukturierte und interaktive Oberfläche direkt im Terminal umgesetzt werden kann. Für die Darstellung von Diagrammen wird `textual-plotext` verwendet. Die Anwendung zeigt unter anderem ein ECTS-Diagramm, eine Notenübersicht über das gesamte Studium, eine Notenübersicht des aktuellen Semesters sowie mehrere Modulbereiche mit Lern- und Prüfungsfortschritt.

Die Daten werden aus einer JSON-Datei importiert, in einer lokalen SQLite-Datenbank gespeichert und beim Start der Anwendung wieder als zusammenhängender Objektgraph geladen. Die Architektur ist in mehrere Verantwortungsbereiche gegliedert: `models.py` enthält das Domänenmodell und die fachliche Logik, `importer.py` übernimmt den Import der Studiendaten, `database.py` kapselt den Zugriff auf SQLite und `main.py` sowie `ui.py` bilden die Benutzeroberfläche. Diese Aufteilung orientiert sich am Single Responsibility Principle, da jede Klasse und Datei eine klar abgegrenzte Aufgabe übernimmt.

4. Ergebnis und Reflexion

Im Ergebnis ist ein lauffähiger Prototyp entstanden, der die im Konzept definierten Ziele technisch umsetzt. Das Dashboard ermöglicht eine schnelle Übersicht über den Studienfortschritt und zeigt, ob der aktuelle Stand im Hinblick auf ECTS-Punkte und Notendurchschnitt zu den gesetzten Zielen passt. Besonders gelungen ist die objektorientierte Struktur des Projekts. Die Trennung zwischen Fachlogik, Datenimport, Persistenz und Darstellung macht den Code nachvollziehbar und erweiterbar.

Besonders positiv war, dass sich die in der Konzeptionsphase geplante Klassenstruktur weitgehend in Python umsetzen ließ. Stolz bin ich vor allem auf die flexible Modellierung der Prüfungsleistungen. Durch die gemeinsame Basisklasse `Exam` und mehrere spezialisierte Unterklassen können unterschiedliche Prüfungsformen abgebildet werden, ohne dass die Benutzeroberfläche jede Prüfungsart einzeln behandeln muss.

Herausfordernd war vor allem die Abstimmung zwischen Datenmodell, Datenbankstruktur und Benutzeroberfläche. Da die Daten zunächst aus einer JSON-Datei importiert, anschließend relational gespeichert und danach wieder als Python-Objekte geladen werden, mussten die Beziehungen zwischen Studiengang, Semestern, Modulen, Prüfungen und Noten korrekt erhalten bleiben. Außerdem musste das Layout der Terminaloberfläche so gestaltet werden, dass Diagramme und Modulbereiche trotz begrenztem Platz übersichtlich bleiben.

Aus dem Projekt wurde deutlich, dass eine klare Trennung der Verantwortlichkeiten die Wartbarkeit verbessert. Gleichzeitig zeigte sich, dass ein fachliches Modell für einen lauffähigen Prototypen um technische Klassen für Import, Persistenz und Darstellung erweitert werden muss. Der Prototyp erfüllt die zentrale Zielsetzung, da Studienfortschritt, Notenentwicklung und Modulstatus übersichtlich dargestellt werden. Er kann als Grundlage für zukünftige Erweiterungen dienen, zum Beispiel für eine direkte Bearbeitung der Studiendaten, zusätzliche Filterfunktionen, Exportfunktionen oder eine spätere Umsetzung als grafische Desktop- oder Webanwendung. Die Ausführung des Prototyps ist in einer Installationsanleitung dokumentiert, sodass das Projekt über das GitHub-Repository nachvollziehbar eingerichtet und getestet werden kann.